

Computing the Context Boundary: A Transaction-Cost Argument for Agent-Aware Services

Author: Fred Smalkin and collaborators

Status: working draft for the economic track of the Context-Boundary Optimization program

Abstract

As software services become larger and more capable, agent consumers face a growing integration problem. A consumer can receive a full capability catalog and construct its own task-specific view, or the service provider can deliver a focused context boundary. This paper argues for provider-side context optimization on transaction-cost grounds. The provider usually holds the lowest-cost position for computing the service view, because it owns the capability map, risk labels, and task paths. A completed large-service efficiency experiment shows that full-catalog exposure scales poorly: combined tokens rose from 4,114 at 10 tools to 22,625 at 200 tools, while completion fell from 0.70 to 0.55. Agent-side retrieval and provider-delivered focused views both held cost near 4,000 tokens and completed more often. The result settles full-catalog dumping as the weaker runtime design in this harness. A follow-up that charged the consumer for its own retrieval closed the provider-versus-consumer comparison under the retrieval-step design tested: priced retrieval scaled with catalog size while the provider-delivered view held flat, so the provider computes the cheaper boundary. An enforcement experiment on an executed-task benchmark adds the authority axis: a consumer-side advisory gate is bypassed as context degrades, from zero to a 0.639 bypass rate, while a provider-enforced gate rejects every violation by construction and raises task completion.

1. The Integration-Cost Problem For Agent-Consumed Services

Agent-consumed services turn integration cost into a first-order product problem. A service can expose a large catalog of actions, schemas, and descriptions, but the consumer still has to decide which small slice matters for a live task. The technical companion paper defines the service boundary problem and reports the first context-contract reliability results [12]. This paper asks who should bear the cost of turning a large service surface into a task-relevant view.

The answer depends on two costs:

- runtime context assembly
- consumer-side development labor

Runtime assembly is paid every time an agent reads, searches, or compresses a service catalog. Development labor is paid when a consumer team builds prompts, wrappers, and policy glue around the service. The economic claim is that both costs fall when the provider computes and ships the context boundary.

2. Runtime Assembly Cost And The Amortization Crossover

Full disclosure looks like the simplest substitute for a context contract: the provider publishes the whole service surface, and each consumer constructs its own optimal view. Disclosure transfers access, while the scarce resource is processing. The costly step is the transformation from raw structure to a task-relevant view, the structural half of the fisheye computation in the technical paper.

Consumer-side construction places that computation inside the bounded context window at inference time. Each task pays to read, search, compress, or infer the relevant service slice. A provider computes the same view offline and amortizes it across consumers and calls. With N consumers running T tasks each, consumer-side construction repeats on the order of $N \times T$ times and repeats again whenever the service changes. Provider-side declaration is one curation pass that updates with the service.

The two arrangements differ in how construction cost scales. Let k be the cost of computing the task-relevant view from the raw service surface, and let e be the cost of applying a precomputed view to one task, with e far below k . Consumer-side construction pays k on every task, for a population total on the order of $N \times T \times k$. Provider-side declaration pays k once and then applies the view at cost e per task, for a total on the order of $k + N \times T \times e$. The provider arrangement wins when $k + N \times T \times e$ falls below $N \times T \times k$, which holds for any service whose view is reused across more than a handful of tasks. The load-bearing assumption is that applying a view costs far less than deriving one, and the large-service experiment supports it: the focused conditions apply a narrow view at flat cost while full-catalog reconstruction climbs with service size.

The crossover prediction follows directly. At small service scope, full disclosure can be cheaper because the catalog is short. As scope grows, the full catalog carries more token cost and more selection risk. A focused view carries setup overhead, then stays close to the task. The completed large-service experiment in Section 7 measures the runtime part of that crossover: full-catalog exposure became more expensive and less reliable as scope grew, while both narrowed conditions stayed close to task size.

The measured numbers ground the model. A two-hundred-tool catalog cost about 22,600 combined tokens per task under full exposure and about 4,000 under a focused view. Full exposure pays the catalog-processing cost on every task, in the role of k ; the focused view applies a narrow, provider-supplied slice, in the role of e . The per-task gap near 18,000 tokens is the amount a consumer population repays on every task when it reconstructs the view in place of receiving it.

3. Consumer-Side Development Cost And The Labor Shift

Runtime cost is only the visible part of integration cost. Consumer teams also pay development labor to make a general agent behave around a specific service, and that labor recurs on every integration:

- mapping operation names to user tasks
- encoding approval and privacy policy
- writing prompt scaffolds and wrappers
- maintaining service-specific examples
- repairing integrations when the service changes

The provider already holds much of the information the consumer reconstructs. Capability grouping, risk labels, prerequisite order, policy boundaries, and common task paths live inside product and platform teams. A context contract turns that internal knowledge into a maintained integration surface.

The labor shift matters because each consumer otherwise rebuilds an incomplete service map. Provider-side context optimization reduces the duplicated work and gives consumers a smaller, more stable contract to integrate. The completed large-service experiment brackets this development cost by giving the retrieval condition a working retrieval layer at no charge. A follow-up should price retrieval-layer construction, prompt scaffolding, and maintenance.

4. Make Or Buy: The Transaction-Cost Argument

Coase frames the firm as a make-or-buy choice: an activity moves inside an organization when internal coordination costs less than repeated market transactions for the same work [4]. Context-boundary computation is such an activity. Turning a large service surface into a task-relevant, governed view is work that someone performs on every integration. The market arrangement has each consumer buy that work by rebuilding the view. The internalized arrangement has the provider make it once and supply it.

Williamson sharpens the choice through asset specificity and transaction frequency: work that depends on specialized, party-held knowledge and repeats across many exchanges aligns with the party that holds the specialized asset [14]. Context-boundary work meets both conditions. The specialized asset is the provider's own structure: its capability map, its risk labels, and its usual task sequences. The frequency is every agent call from every consumer. Both conditions point the activity toward the provider.

The claim is conditional on scale. A small service can publish a flat catalog with little loss, because the view each consumer rebuilds is cheap. A large service with authority-sensitive actions, privacy exposure, or cross-step state raises both the rebuild cost and the coordination risk, so the make arrangement gains. The transaction-cost case therefore predicts a scale threshold rather than a universal rule.

5. Public-Good Under-Provision, Residual-Claimant Incentive, And Enforcement

Consumers underprovide the boundary layer for a standard economic reason. A derived service map is a public good among consumers: once built, it can help other consumers, so no single consumer captures the full value of building and maintaining it. A consumer-built map also binds nobody. It can advise an agent, but it has limited authority over provider behavior.

The provider is the residual claimant on integration quality. Better agent task success raises the service's value, reduces support friction, and improves adoption. The provider is also the lowest-cost producer of the layer, because the capability map, risk labels, and task sequences already exist inside the service.

Where providers abstain, intermediaries sell the missing layer. The market already has API aggregators, wrapper SDKs, community interface specifications, and curated tool registries. Intermediary provision confirms demand for the boundary layer. It also trails provider provision on enforcement: an intermediary can describe an approval gate, while only the service can enforce one.

Enforcement is the sharpest form of the answer. A context contract is a commitment, and disclosure is data. Authority gates, provenance requirements, and memory rules bind behavior when the service declares and enforces them. Consumer-side inference can approximate the provider's structure; provider-side declaration can issue guarantees.

An enforcement experiment on tau2-bench measures this gap directly. The same agent ran executed service tasks under two regimes: an advisory gate, where the domain policy sat in the agent's prompt and the environment executed whatever irreversible write the agent issued, and a provider-enforced gate, where a deterministic wrapper rejected any write whose policy precondition was unmet before it executed. Advisory compliance depended on the agent's context. With the full policy in context, the agent bypassed no write gate across 55 attempts. With the policy stripped, it bypassed 46 of 72 write attempts, a 0.639 bypass rate. The provider gate caught every one of those violations by construction. A live enforced run on the cells where advisory bypass was near-total confirmed the guarantee at execution time: no violation reached execution, the agent adapted to each rejection and satisfied the precondition before proceeding, and native completion on those cells rose from 0.533 under advisory to 0.867 under enforcement, at the cost of a few extra turns.

Enforcement here bought capability. The structural gate closed the bypass gap and raised completion together, because forcing the agent to meet the obligation it skipped also pushed it onto the path that completed the task. Advisory compliance is a property of the agent and its context; provider enforcement is a property of the service, which is why only the service can guarantee it. docs/tau2-bench-integration.md and experiments/tau2-enforcement/ hold the

setup and results.

6. Market-Structure Prediction: Service Scope Inverse To Consumer Effort

Services will grow broader and more capable, while the consumer work required per task should fall. The cost of that simplicity moves to the supplier side.

This prediction has three observable signs:

- providers ship focused service views alongside broad catalogs
- consumers supply less service-specific prompt scaffolding over time
- intermediaries narrow toward discovery, comparison, and compatibility layers

MCP is a useful connection layer for this structure. It standardizes how agent applications discover and call tools. The economic question is what the MCP server should carry once the connection exists. The prediction says larger services will carry more context-boundary work on the server side, because consumer-side reconstruction becomes too expensive at scale.

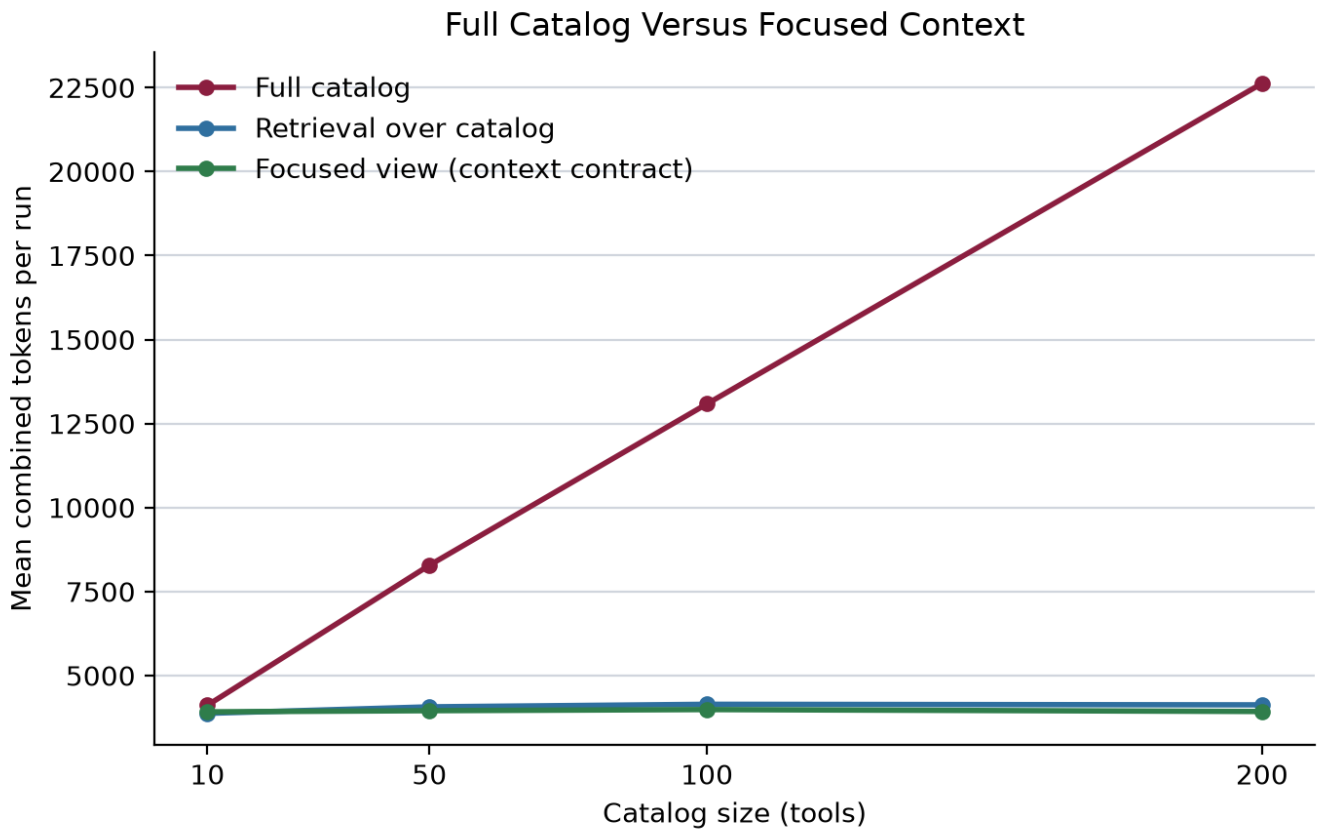
7. Empirical Illustration: The Large-Service Efficiency Experiment

The large-service efficiency experiment tested the runtime crossover directly. It used a synthetic service with multiple capability groups. Catalog size varied across 10, 50, 100, and 200 tools. Each task needed a small tool subset. The 240 runs crossed `full_catalog`, `retrieval_over_catalog`, and `focused_view` conditions. The result record is `docs/large-service-results.md`.

Combined agent-and-evaluator tokens by catalog size:

Catalog size	<code>full_catalog</code>	<code>retrieval_over_catalog</code>	<code>focused_view</code>
10	4,114	3,875	3,920
50	8,283	4,068	3,954
100	13,085	4,145	3,987
200	22,625	4,128	3,930

The crossover appeared on the full-catalog path. `full_catalog` rose from 4,114 to 22,625 combined tokens, about 5.5x, and completion fell from 0.70 to 0.55. `retrieval_over_catalog` and `focused_view` both held near 4,000 combined tokens. Their completion rates stayed above `full_catalog` at every size.

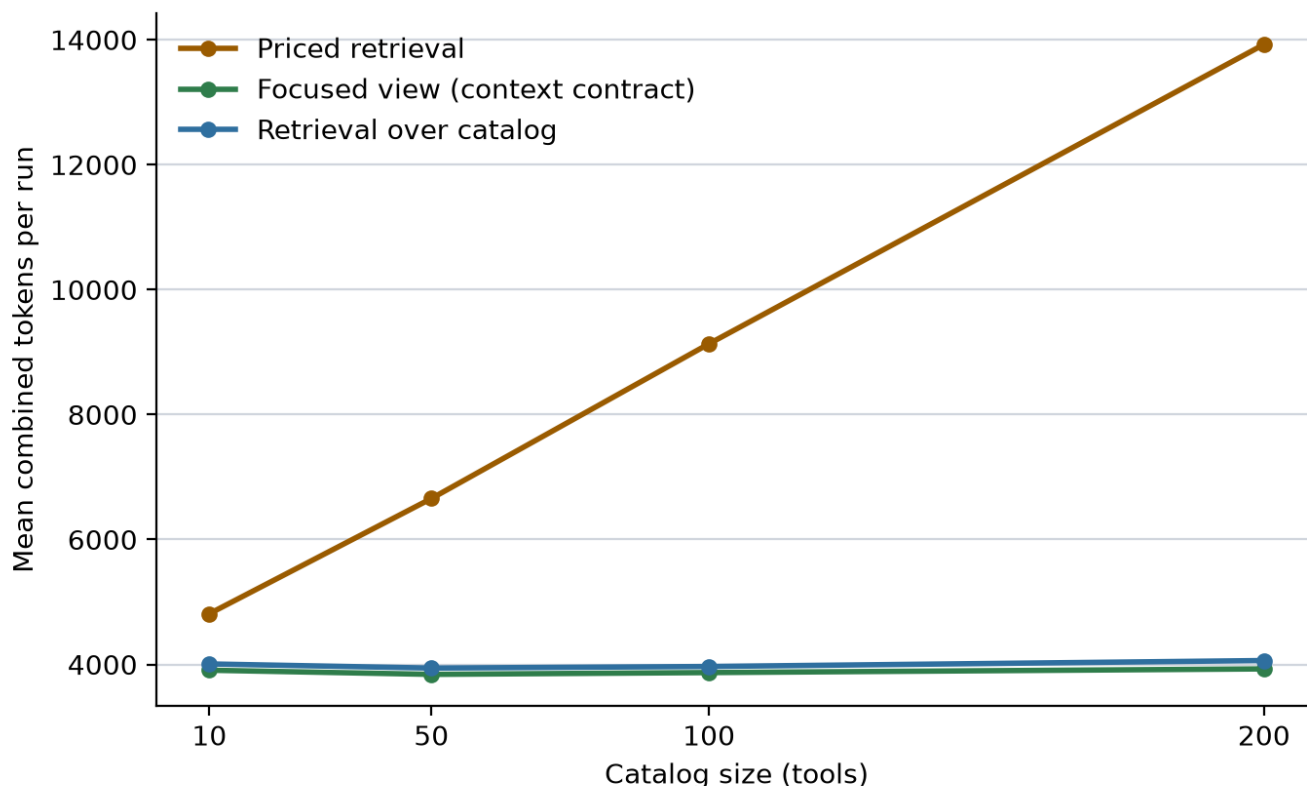


Combined tokens by catalog size under full-catalog exposure, agent-side retrieval, and a provider-delivered focused view. The full catalog scales about 5.5 times across the range while both narrowed conditions stay near 4,000 tokens.

This result confirms the amortization argument for full-catalog dumping. Runtime cost and completion settle dump-versus-focused selection. They leave provider-versus-consumer focus open because `retrieval_over_catalog` matched `focused_view` on tokens and completion, and retrieval carried fewer critical failures. The provider-versus-consumer choice therefore rests on the development-cost argument in Section 3 and the enforcement argument in Section 5.

A follow-up run priced the retrieval layer. In the `retrieval_priced` condition the agent reads the catalog and selects its tools in one model call, then acts in a second, and the consumer pays both. Priced retrieval scaled with catalog size, from about 4,800 combined tokens at ten tools to about 13,900 at two hundred, while the provider-delivered focused view held near 3,900 at every size and completed more often. The free-retrieval condition stayed near 4,000, so the earlier tie came from handing the consumer a retrieval layer at no charge. Charging retrieval its own cost closes the provider-versus-consumer comparison under this retrieval-step design: consumer-side selection grows with the service it must search, while the provider computes the view once and delivers a flat slice. A development-cost model over consumer count N adds the setup dimension, and provider-side delivery is cheaper across the tested range, with the runtime gap alone favoring the provider at large catalog size even for one consumer.

Priced Retrieval Versus Focused Context



Combined tokens by catalog size when the consumer pays for its own retrieval. Priced retrieval climbs from about 4,800 tokens at 10 tools to about 13,900 at 200, while the provider-delivered focused view and the free-retrieval condition stay near 4,000.

The Cost-Capability Spectrum

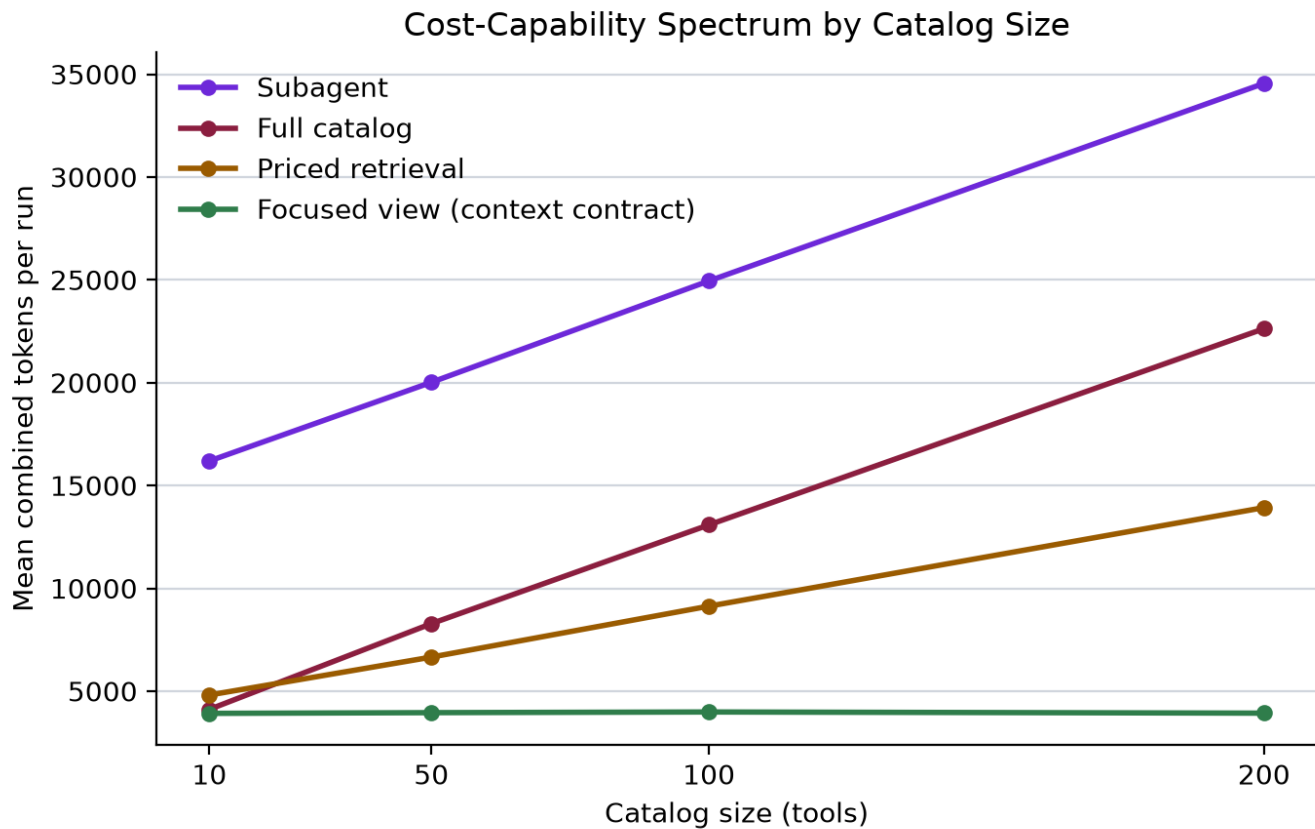
Priced retrieval is a minimal consumer-side agent: two model calls, select then act. A full subagent does more. A subagent condition ran a four-call reasoning loop over the full catalog for each task, surveying the catalog, selecting tools and obligations, reflecting for omissions, then acting, with the consumer paying every call. It ran at all four catalog sizes with the same tasks and trials.

Catalog size	Subagent tokens	Subagent completion	Focused-view tokens
10	16,192	1.00	3,907
50	20,018	1.00	3,841
100	24,948	1.00	3,869
200	34,552	1.00	3,928

The subagent completed every task at every size with near-zero critical failures, the strongest capability of any condition. It also cost the most and scaled worst: combined tokens rose from 16,192 at ten tools to 34,552 at two hundred, four to nine times the flat focused-view cost, and the gap widened with the catalog. Latency tracks the same axis through model round-trips, which set the wall-clock floor: the focused view is one round-trip, priced retrieval is two, and the subagent is four, so the subagent pays about nine times the focused view's measured latency.

The three consumer-side options therefore trace a cost-capability spectrum. A raw catalog is cheap to expose and degrades on cost and reliability as the service grows. A subagent that rebuilds the task-scoped boundary by reasoning reaches the highest capability and pays for it in tokens and round-trips that scale with the service. The provider-delivered context contract sits between them: near-subagent completion at close to raw-catalog cost and a single round-trip, because

the provider computes the view once rather than each consumer reconstructing it per call. The transaction-cost claim is that this middle position is the low-cost producer's, with a subagent reserved for the long tail of tasks beyond a precomputed contract's coverage.



Consumer-side options on a cost-capability spectrum by catalog size. The subagent reaches the highest completion but its cost climbs from about 16,000 to 35,000 tokens across the range, while the provider-delivered focused view holds near 4,000 at one model round-trip against the subagent's four.

8. Implications For Platform Strategy, Pricing, And Standards

The strategy implication follows from the market-structure prediction rather than from the experiment alone. A service that expects agent consumption should budget for boundary design as part of the product surface. A large raw API catalog shifts too much selection work to consumers. A maintained context contract turns service knowledge into an integration asset.

The pricing implication follows the same logic. If provider-side context optimization reduces repeated consumer work, providers can price the service as a lower-integration-cost product. The value claim centers on reduced consumer engineering effort and lower runtime context cost; task completion remains a technical-track measure.

Standards matter because boundary work needs a carrier. MCP can carry the connection, the tool schema, and the resource surface. Context contracts can define the task-scoped view, approval obligations, provenance requirements, and output record that travel over that connection. The platform opportunity is to make those terms portable enough for consumers while preserving provider enforcement.

9. Related Work

The argument draws on three literatures: platform boundary resources, two-sided-market economics, and interface design.

Platform research treats the interface a provider exposes as a governed artifact. Ghazawneh and Henfridsson model boundary resources, the software and documentation a platform supplies so third-party developers can build on it while the platform keeps control [6]. Eaton and colleagues show that tuning those boundary resources is ongoing, distributed work between the platform and its developers [5]. Tiwana, Konsynski, and Bush frame platform architecture and governance as a provider design choice [13], and Boudreau shows that granting access to a platform interface shifts who innovates and how much [3]. Benzell, Hersh, and Van Alstyne link opening an interface through public APIs to measured firm growth, an inverted-firm strategy where outside developers build on the provider's capabilities [2]. A context contract is a boundary resource for agent developers, and this paper adds the transaction-cost account of why the provider, in place of each consumer, should compute it.

Two-sided-market and ecosystem economics explain why a platform absorbs cost to grow its complementor base. Rochet and Tirole formalize how a platform sets terms across both sides of a market [11]. Jacobides, Cennamo, and Gawer describe how ecosystems align complementors around a focal offering [7]. The market-structure prediction here, that service scope grows while consumer integration effort per task falls, is a specific claim in this tradition, focused on the integration and context cost in place of pricing or subsidy.

Interface design supplies the older half of the argument. Parnas grounds the interface in information hiding, exposing what a caller needs and concealing the rest [10]. Liskov and Zilles cast the interface as a behavioral contract independent of implementation [9]. Lieberherr and Holland state the law of least knowledge, a caller-side limit on how much of a provider's structure a call should reach [8]. Baldwin and Clark tie modular interfaces to economic value [1]. A focused service view carries these principles to an AI-agent caller and a task-scoped, runtime boundary, and the companion technical paper develops its fisheye form [12].

These literatures treat the boundary as a governance artifact, a strategy variable, or a static software interface. The make-or-buy choice at the interface itself, who should compute the runtime context boundary for autonomous-agent consumers, remains open, and this paper takes it up with a transaction-cost argument and an efficiency prediction the large-service experiment tests.

10. Limitations

The argument is analytic. It applies transaction-cost reasoning to service boundaries and predicts where the cost should settle. The paper does not yet measure real integration labor across providers and consumers.

The large-service experiment is one completed illustration inside a synthetic service. It settles runtime dump-versus-focused cost and reliability for this harness. A priced-retrieval follow-up then charged the consumer for its retrieval step and measured a development-cost crossover, so the provider-versus-consumer comparison now rests on a specific retrieval-step operationalization that future work can vary. The enforcement experiment uses one agent model and a policy-precondition gate derived from the tau2-bench domains, so its bypass and completion figures are specific to that substrate; the structural point, that only the service can guarantee rejection, holds by construction. Market adoption, pricing, and standards outcomes require external evidence.

The market read is a projection. Provider-side contracts can become stale, incomplete, or self-serving. Consumers still need verification tools, contract provenance, and fallback access to broader service views. Those governance requirements sit alongside the economic case rather than outside it.

References

1. Baldwin, C. Y., & Clark, K. B. (2000). *Design Rules, Volume 1: The Power of Modularity*. MIT Press. ISBN 978-0-262-02466-2.
2. Benzell, S. G., Hersh, J., & Van Alstyne, M. (2024). *How APIs Create Growth by Inverting the Firm*. *Management Science*, 70(10), 7120-7141. DOI:10.1287/mnsc.2023.4968.
3. Boudreau, K. J. (2010). *Open Platform Strategies and Innovation: Granting Access Versus Devolving Control*. *Management Science*, 56(10), 1849-1872. DOI:10.1287/mnsc.1100.1215.
4. Coase, R. H. (1937). *The Nature of the Firm*. *Economica, New Series*, 4(16), 386-405. DOI:10.1111/j.1468-0335.1937.tb00002.x. <https://onlinelibrary.wiley.com/doi/full/10.1111/j.1468-0335.1937.tb00002.x>
5. Eaton, B., Elaluf-Calderwood, S., Sorensen, C., & Yoo, Y. (2015). *Distributed Tuning of Boundary Resources: The Case of Apple's iOS Service System*. *MIS Quarterly*, 39(1), 217-243. DOI:10.25300/MISQ/2015/39.1.10.
6. Ghazawneh, A., & Henfridsson, O. (2013). *Balancing platform control and external contribution in third-party development: the boundary resources model*. *Information Systems Journal*, 23(2), 173-192. DOI:10.1111/j.1365-2575.2012.00406.x.
7. Jacobides, M. G., Cennamo, C., & Gawer, A. (2018). *Towards a theory of ecosystems*. *Strategic Management Journal*, 39(8), 2255-2276. DOI:10.1002/smj.2904.
8. Lieberherr, K. J., & Holland, I. M. (1989). *Assuring Good Style for Object-Oriented Programs*. *IEEE Software*, 6(5), 38-48. DOI:10.1109/52.35588.
9. Liskov, B., & Zilles, S. (1974). *Programming with Abstract Data Types*. *ACM SIGPLAN Notices*, 9(4), 50-59. DOI:10.1145/942572.807045.
10. Parnas, D. L. (1972). *On the Criteria To Be Used in Decomposing Systems into Modules*. *Communications of the ACM*, 15(12), 1053-1058. DOI:10.1145/361598.361623.
11. Rochet, J.-C., & Tirole, J. (2003). *Platform Competition in Two-Sided Markets*. *Journal of the European Economic Association*, 1(4), 990-1029. DOI:10.1162/154247603322493212.
12. Smalkin, F., & collaborators. (2026). *Context-Boundary Optimization for Agent-Aware Services*. Companion technical paper, working draft, drafts/technical-paper.md.
13. Tiwana, A., Konsynski, B., & Bush, A. A. (2010). *Platform Evolution: Coevolution of Platform Architecture, Governance, and Environmental Dynamics*. *Information Systems Research*, 21(4), 675-687. DOI:10.1287/isre.1100.0323.
14. Williamson, O. E. (1979). *Transaction-Cost Economics: The Governance of Contractual Relations*. *Journal of Law and Economics*, 22(2), 233-261. DOI:10.1086/466942. <https://www.journals.uchicago.edu/doi/10.1086/466942>